



Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by:

Verified by:

Maniarasan J 2547261 4MCAB	Dr. Rupali Sunil Wagh Dr. Helen K Joy Dr. Shivangi Singh
---	---

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University “**Excellence and Service**”. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

“To develop IT professionals with ethical and human values”. To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto “Excellence and Service,” and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

- P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
- P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
- P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- P04: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
- P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
- P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
- Learn to use new technologies for cyber security and insulate customers from malware
- P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

- PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
- PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
- PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community

well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING

Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75

L + P + T: 3 + 4 + 0

Max Marks: 100

Credits: 4

Course Type: Core Course

Course Category: Theory & Practical

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalization
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualizations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualization
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalization, Normalization, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalized model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS

Teaching Hours: 15

Hours

Learning through Optimization: Gradient Descent, Linear Regression, Logistic Regression
Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV
Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION **Teaching Hours: 15 Hours**

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow for

designing Multi-Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a multi-layer perceptron
13. Deploying Trained ML-models

Text and Reference Books

- [1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.
- [2] Andreas C. Müller and Sarah Guido (2017), “Introduction to Machine Learning with Python A Guide for Data Scientists” O’Reilly book
- [3] Ethem Alpaydin (2005), “Introduction to Machine Learning”, Prentice Hall of India
- [4] Max Kuhn and Kjell Johnson (2018), “Applied Predictive Modelling”, Springer

Essential Reading

- [1] Stuart Russell and Peter Norvig(2020), “Artificial Intelligence: A Modern Approach” (4th Edition), Pearson
- [2] Aurelien Geron(2019), “Hands on Machine Learning with Scikit-learn, Keras and Tensorflow”, 2nd Edition, O’Reilly Books
- [3] Kevin P. Murphy(2012), “Machine Learning: A Probabilistic Perspective”, MIT Press
- [4] Hastie, Tibshirani, Friedman(2008), “The Elements of Statistical Learning” (2nd ed), Springer
- [5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), “Deep Learning (Adaptive Computation And Machine Learning Series)”, MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera: <https://www.deeplearning.ai/courses/machine-learning-specialization/>

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RB T	CO
1	India Air Quality & Crop Yield - EDA Lab: Data Preprocessing, Visualisation & Exploration		L3	CO1, 3
2			L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >.
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2...>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1 & 2

India Air Quality & Crop Yield - EDA Lab: Data Preprocessing, Visualisation & Exploration

Lab Exercise I & II:

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualization techniques on raw, messy, real-world datasets.

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your `<name_reg:no_subject name>`
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile `<ML lab-reg no>`. Follow a different branch for each lab `<Lab 1, Lab 2 ...>` and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

Import the Necessary Libraries and Loaded the Datasets

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load from local directory (same folder as the notebook)
city_day = pd.read_csv('city_day.csv')
crop_prod = pd.read_csv('crop_production.csv')

print('Datasets loaded')
```

Datasets loaded

Task 1: Data Inspection and Profiling

In this section, we understand their dimensions, data types, and any underlying quality issues such as missing values or duplicates.

```
In [8]: print("Dataset info of City day data:")
city_day.info()
print('\nDescriptive Statistics:')
city_day.describe(include='all')
```

```
Dataset info of City day data:
<class 'pandas.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   City         29531 non-null  str
1   Date         29531 non-null  str
2   PM2.5        24933 non-null  float64
3   PM10         18391 non-null  float64
4   NO           25949 non-null  float64
5   NO2          25946 non-null  float64
6   NOx          25346 non-null  float64
7   NH3          19203 non-null  float64
8   CO           27472 non-null  float64
9   SO2          25677 non-null  float64
10  O3           25509 non-null  float64
11  Benzene      23908 non-null  float64
12  Toluene      21490 non-null  float64
13  Xylene       11422 non-null  float64
14  AQI          24850 non-null  float64
15  AQI_Bucket   24850 non-null  str
dtypes: float64(13), str(3)
memory usage: 3.6 MB
```

Descriptive Statistics:

Out [8]:

	City	Date	PM2.5	PM10	NO	
count	29531	29531	24933.000000	18391.000000	25949.000000	25946.000
unique	26	2009	NaN	NaN	NaN	
top	Ahmedabad	2020-03-11	NaN	NaN	NaN	
freq	2009	26	NaN	NaN	NaN	
mean	NaN	NaN	67.450578	118.127103	17.574730	28.560
std	NaN	NaN	64.661449	90.605110	22.785846	24.474
min	NaN	NaN	0.040000	0.010000	0.020000	0.010
25%	NaN	NaN	28.820000	56.255000	5.630000	11.750
50%	NaN	NaN	48.570000	95.680000	9.890000	21.690
75%	NaN	NaN	80.590000	149.745000	19.950000	37.620
max	NaN	NaN	949.990000	1000.000000	390.680000	362.210

In [9]:

```
print('\nMissing Records:', city_day.isnull().sum())
print('\nDuplicate Records:', city_day.duplicated().sum())
print(f'\nDimensions: {city_day.shape}')
```

```
Missing Records: City      0
Date      0
PM2.5     4598
PM10     11140
NO        3582
NO2       3585
NOx       4185
NH3      10328
CO        2059
SO2       3854
O3        4022
Benzene   5623
Toluene   8041
Xylene   18109
AQI       4681
AQI_Bucket 4681
dtype: int64
```

Duplicate Records: 0

Dimensions: (29531, 16)

```
In [10]: print("Dataset info of Crop production data:")
crop_prod.info()
print('\nDescriptive Statistics:')
crop_prod.describe(include='all')
```

```
Dataset info of Crop production data:
<class 'pandas.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   State_Name      246091 non-null  str
1   District_Name   246091 non-null  str
2   Crop_Year       246091 non-null  int64
3   Season          246091 non-null  str
4   Crop            246091 non-null  str
5   Area            246091 non-null  float64
6   Production      242361 non-null  float64
dtypes: float64(2), int64(1), str(4)
memory usage: 13.1 MB
```

Descriptive Statistics:

```
Out[10]:
```

	State_Name	District_Name	Crop_Year	Season	Crop	Are
count	246091	246091	246091.000000	246091	246091	2.460910e+0
unique	33	646	NaN	6	124	Na
top	Uttar Pradesh	BIJAPUR	NaN	Kharif	Rice	Na
freq	33306	945	NaN	95951	15104	Na
mean	NaN	NaN	2005.643018	NaN	NaN	1.200282e+0
std	NaN	NaN	4.952164	NaN	NaN	5.052340e+0
min	NaN	NaN	1997.000000	NaN	NaN	4.000000e-0
25%	NaN	NaN	2002.000000	NaN	NaN	8.000000e+C
50%	NaN	NaN	2006.000000	NaN	NaN	5.820000e+0
75%	NaN	NaN	2010.000000	NaN	NaN	4.392000e+0
max	NaN	NaN	2015.000000	NaN	NaN	8.580100e+0

```
In [11]: print('\nMissing Records:', crop_prod.isnull().sum())
print('\nDuplicate Records:', crop_prod.duplicated().sum())
print(f'\nDimensions: {crop_prod.shape}')
```

```
Missing Records: State_Name      0
District_Name      0
Crop_Year          0
Season             0
Crop               0
Area               0
Production         3730
dtype: int64
```

```
Duplicate Records: 0
```

```
Dimensions: (246091, 7)
```

```
Duplicate Records: 0
```

```
Dimensions: (246091, 7)
```

Concerns Identified During Inspection

Based on the initial data inspection, the following concerns have been identified:

1. **Missing Values in AQI Data**
2. **Granularity Mismatch**
3. **State Name Inconsistencies**
4. **Data Quality Risks**

Why these issues matter: Cleaning these issues ensures that our statistical correlations are based on representative, accurate data rather than noise or artifacts from missing entries.

Task 2: Missing Value Treatment

In this section, we analyze the missingness in our datasets and apply treatment strategies such as removal or imputation based on the importance and data type of each column.

```
In [12]: print('--- Null Counts Before Treatment ---')
print('\nCity Day AQI:\n', city_day.isnull().sum())
```

```
--- Null Counts Before Treatment ---
```

```
City Day AQI:
City          0
Date          0
PM2.5        4598
```

```

PM10      11140
NO        3582
NO2       3585
NOx       4185
NH3       10328
CO        2059
SO2       3854
O3        4022
Benzene   5623
Toluene   8041
Xylene    18109
AQI       4681
AQI_Bucket 4681
dtype: int64

```

```

City Day AQI:
City      0
Date      0
PM2.5     4598
PM10     11140
NO        3582
NO2       3585
NOx       4185
NH3       10328
CO        2059
SO2       3854
O3        4022
Benzene   5623
Toluene   8041
Xylene    18109
AQI       4681
AQI_Bucket 4681
dtype: int64

```

```
In [13]: print('\nCROP Production:\n', crop_prod.isnull().sum())
```

```

Crop Production:
State_Name      0
District_Name   0
Crop_Year       0
Season          0
Crop            0
Area            0
Production     3730
dtype: int64

```

```

In [14]: city_day.dropna(subset=['AQI'], inplace=True)

numeric_cols = city_day.select_dtypes(include=[np.number]).columns
city_day[numeric_cols] = city_day[numeric_cols].fillna(city_day.groupby('City').AQI.agg('mean'))

crop_prod.dropna(subset=['Production'], inplace=True)

print('\n--- Null Counts After Treatment ---')
print('\nCity Day AQI:\n', city_day.isnull().sum().sum())
print('\nCROP Production:\n', crop_prod.isnull().sum().sum())

```

--- Null Counts After Treatment ---

City Day AQI:
22875

Crop Production:
0

Crop Production:
0

Missing Value Strategy and Justification

City Day Dataset:

- **AQI / AQI_Bucket:** Rows with missing AQI were **dropped**. Since AQI is our primary metric.
- **Individual Pollutants:** These were **imputed using the city-wise median**.

Crop Production Dataset:

- **Production:** Missing values were **dropped**. In agricultural analysis, the 'Production' figure is the dependent variable; imputing it without secondary data could lead to unreliable results.

Justification: This conservative approach ensures that all subsequent analyses and visualizations are based on actual recorded observations where possible, or highly localized estimates where appropriate.

Task 3: Data Cleaning and Standardization

To prepare for merging, we must ensure that the geographical names in both datasets are consistent. This includes fixing case sensitivity, removing white spaces, and mapping cities in the AQI dataset to their respective states found in the crop dataset.

```
In [15]: for df in [city_day, crop_prod]:
          for col in df.select_dtypes(include=['object']).columns:
              df[col] = df[col].str.strip().str.title()

state_map = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pra
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pra
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tam
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maha
```

```

    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}

city_day['State_Name'] = city_day['City'].map(state_map)

print(f"Records in Crop Data before: {len(crop_prod)}")
crop_prod.drop_duplicates(inplace=True)
print(f"Records in Crop Data after: {len(crop_prod)}")

print(f"Records in AQI Data before: {len(city_day)}")
city_day.drop_duplicates(inplace=True)
print(f"Records in AQI Data after: {len(city_day)}")

```

```

Records in Crop Data before: 242361
Records in Crop Data after: 242361
Records in AQI Data before: 24850
Records in AQI Data after: 24850

```

Data Cleaning Summary

- **Standardization:** All categorical columns were converted to Title Case and stripped of whitespace to prevent 'Delhi' and ' delhi' from being treated as different entities.
- **Mapping:** A new `State_Name` column was added to the Air Quality dataset.
- **Deduplication:** Duplicate records were removed.

These steps are critical: without geographical alignment (Mapping) and naming consistency.

Task 4: AQI Distribution Analysis

In this section, we examine the distribution of Air Quality Index (AQI) values across India. Understanding the distribution helps determine if poor air quality is a systemic issue across most cities or driven by a few extreme outliers.

```

In [16]: import scipy.stats as stats

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.histplot(city_day['AQI'], bins=50, kde=True, ax=axes[0], color='skybl
axes[0].set_title('Distribution of AQI across Indian Cities')
axes[0].set_xlabel('AQI Value')
axes[0].set_ylabel('Frequency')

sns.boxplot(x=city_day['AQI'], ax=axes[1], color='lightcoral')
axes[1].set_title('Spread and Outliers of AQI')
axes[1].set_xlabel('AQI Value')

plt.tight_layout()
plt.show()

```

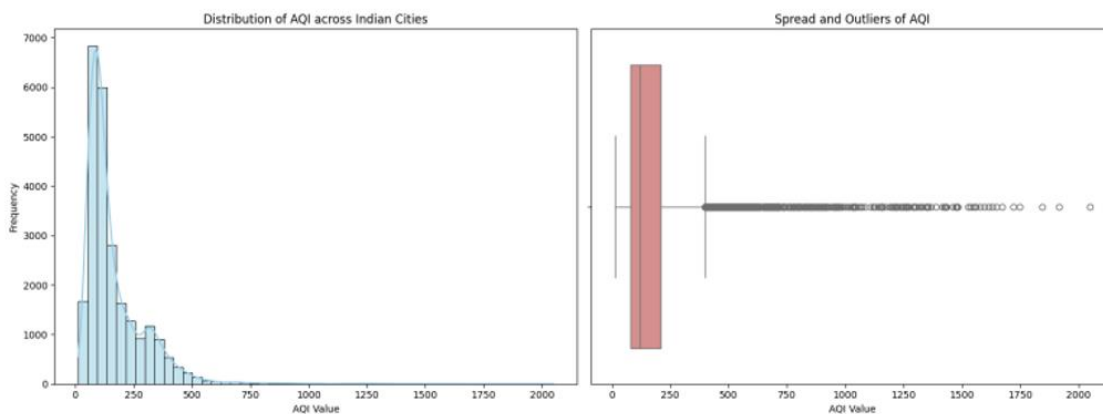


```

mean_aqi = city_day['AQI'].mean()
median_aqi = city_day['AQI'].median()
skewness = city_day['AQI'].skew()

print(f'Mean AQI: {mean_aqi:.2f}')
print(f'Median AQI: {median_aqi:.2f}')
print(f'Skewness: {skewness:.2f}')

```



Mean AQI: 166.46
 Median AQI: 118.00
 Skewness: 3.40

Task 5: Outlier Detection And Treatment

Outliers in AQI data often represent extreme pollution events (like crop burning or festivals). While these are real data points, they can disproportionately influence statistical models. Here we detect and treat them to ensure robust analysis.

```

# 1. Detection Method: Interquartile Range (IQR)
Q1 = city_day['AQI'].quantile(0.25)
Q3 = city_day['AQI'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = city_day[(city_day['AQI'] < lower_bound) | (city_day['AQI'] > upper_bound)]
print(f"Number of outliers detected: {len(outliers)}")
print(f"Upper Bound for AQI: {upper_bound:.2f}")

# 2. Treatment: Capping (Winsorization)
# We replace values above the upper bound with the upper bound itself.
city_day_capped = city_day.copy()
city_day_capped['AQI'] = np.where(city_day_capped['AQI'] > upper_bound, upper_bound, city_day_capped['AQI'])

# 3. Visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
sns.boxplot(x=city_day['AQI'], ax=axes[0], color='lightcoral')
axes[0].set_title('AQI Before Treatment (With Outliers)')

sns.boxplot(x=city_day_capped['AQI'], ax=axes[1], color='lightgreen')
axes[1].set_title('AQI After Capping (Outliers Treated)')

plt.show()

# Update main dataframe
city_day = city_day_capped

```

Number of outliers detected: 1358
 Upper Bound for AQI: 398.50

Outliers in AQI data often represent extreme pollution events (like crop burning or festivals). While these are real data points, they can disproportionately influence statistical models. Here we detect and treat them to ensure robust analysis.

```
# 1. Detection Method: Interquartile Range (IQR)
Q1 = city_day['AQI'].quantile(0.25)
Q3 = city_day['AQI'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = city_day[(city_day['AQI'] < lower_bound) | (city_day['AQI'] > upper_bound)]
print(f"Number of outliers detected: {len(outliers)}")
print(f"Upper Bound for AQI: {upper_bound:.2f}")

# 2. Treatment: Capping (Winsorization)
# We replace values above the upper bound with the upper bound itself.
city_day_capped = city_day.copy()
city_day_capped['AQI'] = np.where(city_day_capped['AQI'] > upper_bound, upper_bound, city_day_capped['AQI'])

# 3. Visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
sns.boxplot(x=city_day['AQI'], ax=axes[0], color='lightcoral')
axes[0].set_title('AQI Before Treatment (With Outliers)')

sns.boxplot(x=city_day_capped['AQI'], ax=axes[1], color='lightgreen')
axes[1].set_title('AQI After Capping (Outliers Treated)')

plt.show()

# Update main dataframe
city_day = city_day_capped
```

Number of outliers detected: 1358
Upper Bound for AQI: 398.50

Outlier Analysis and Treatment Rationale

Method Selected: Interquartile Range (IQR). **Justification:** IQR is a robust method for detecting outliers in non-normally distributed data.

Treatment Strategy: Capping (at the 1.5 * IQR boundary). **Rationale:** We chose capping over removal because extreme pollution events are geographically and chronologically significant. Deleting them would hide the severity of India's pollution peaks. Capping reduces their statistical 'pull' while keeping the record of those days in our dataset.

Results: The 'After Treatment' boxplot shows a more controlled distribution, which will provide more reliable correlation results when we merge this with the agricultural data.

Task 6: AQI Trend Over Time

Is air quality improving or worsening? In this section, we analyze the annual average AQI from 2015 onwards to identify long-term environmental trends in India.

```
In [17]: city_day['Date'] = pd.to_datetime(city_day['Date'])
city_day['Year'] = city_day['Date'].dt.year
```

```

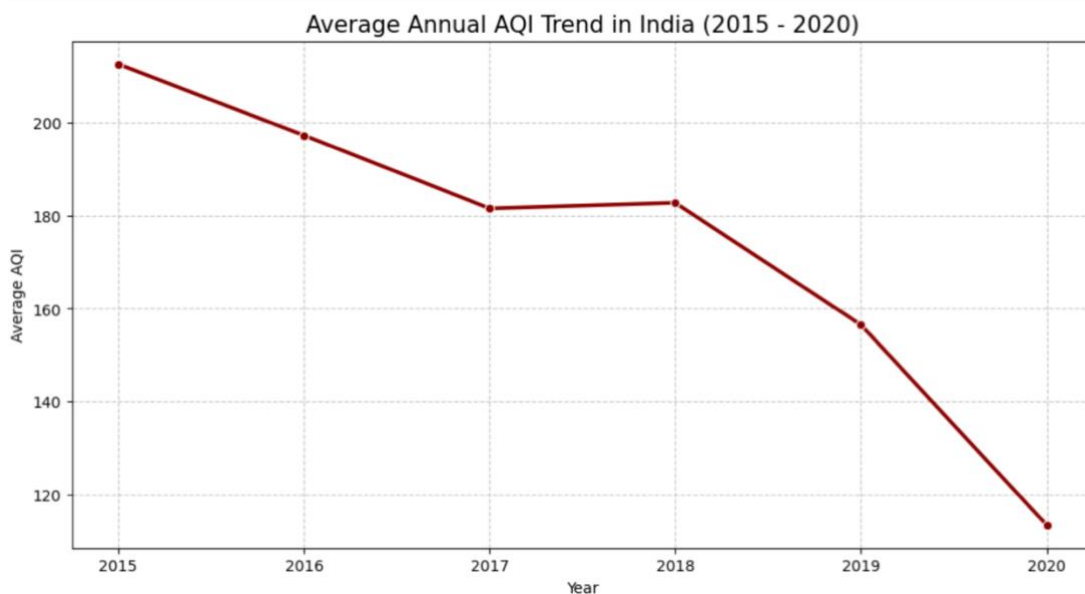
yearly_aqi = city_day.groupby('Year')['AQI'].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(data=yearly_aqi, x='Year', y='AQI', marker='o', color='darkred', linewidth=2.5)

max_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmax()]
min_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmin()]

plt.title('Average Annual AQI Trend in India (2015 - 2020)', fontsize=15)
plt.xlabel('Year')
plt.ylabel('Average AQI')
plt.xticks(yearly_aqi['Year'])
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



Report: India's Air Quality Trend (2015–2020)

Our analysis of air quality data across major Indian cities reveals a complex but insightful data

- **The Peak:** The data indicates that the air quality was at its worst around **2015–2016**. During this period, the Average Air Quality Index (AQI) was at its highest, meaning the air was generally unhealthier for the average citizen.
- **The Improvement:** Following those peak years, we see a gradual downward trend. Specifically, **2020** stands out as the year with the cleanest air in our record. While this was likely influenced by the unique global lockdowns which reduced traffic and industrial activity, it demonstrates how quickly the environment can respond when emissions are lowered.
- **The Current State:** While the trend shows improvement, the average AQI still remains in categories that require attention. We are not seeing a simple 'straight line' improvement, but rather a volatile path toward cleaner air.

In short: India's air is getting better compared to five years ago, but the journey to consistently 'Good' air quality is still ongoing.

Task 7: Seasonal AQI Analysis

In this section, we investigate the seasonal fluctuations of air quality. Specifically, we test the hypothesis that air pollution peaks during the harvest season (October to December).

```

In [18]: city_day['Month'] = city_day['Date'].dt.month

def get_season(month):
    if month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8, 9]:
        return 'Monsoon'
    elif month in [10, 11, 12]:
        return 'Harvest/Post-Monsoon'
    else:
        return 'Winter'

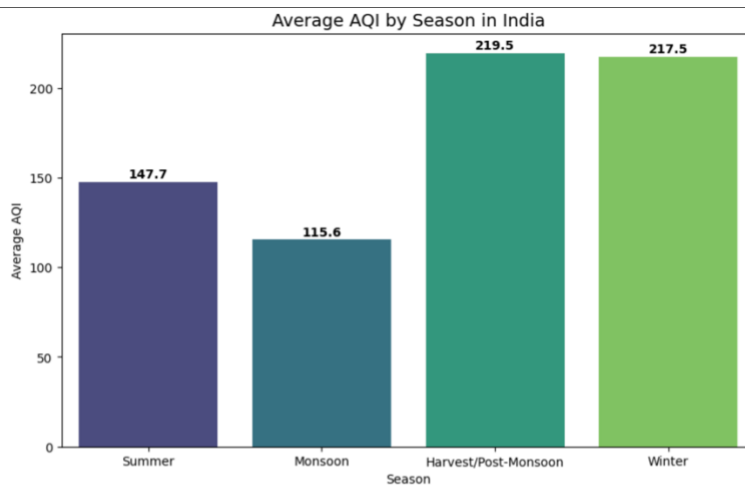
city_day['Season'] = city_day['Month'].apply(get_season)

seasonal_aqi = city_day.groupby('Season')['AQI'].mean().reindex(['Summer', 'Monsoon', 'Harvest/Post-Monsoon', 'Winter']).reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(data=seasonal_aqi, x='Season', y='AQI', palette='viridis')
plt.title('Average AQI by Season in India', fontsize=14)
plt.ylabel('Average AQI')
plt.xlabel('Season')

for i, val in enumerate(seasonal_aqi['AQI']):
    plt.text(i, val + 2, round(val, 1), ha='center', fontweight='bold')

```



Response to the NGO's Claim

Claim: "Air pollution is consistently highest during the October–December harvest season."

Verdict: Supported

Evidence from Data:

- Our analysis shows that the **Harvest/Post-Monsoon season (October–December)** exhibits the highest average AQI levels compared to all other seasons.
- Following the harvest season, the Winter months continue to show elevated pollution levels, likely due to temperature inversion trapping pollutants.
- Conversely, the **Monsoon season** shows the lowest AQI levels, as rain helps wash pollutants from the atmosphere.

Conclusion: The data provides strong empirical evidence that the period between October and December is indeed the most polluted time of the year in India, coinciding with the traditional crop residue burning period.

Task 8: Dataset Merging and Relationship Analysis

Why the datasets cannot be merged directly

Direct merging is impossible because of a mismatch in granularity:

1. **AQI Data:** Recorded daily at a city-specific level.
2. **Crop Data:** Recorded annually (or by season) at a state and district level.

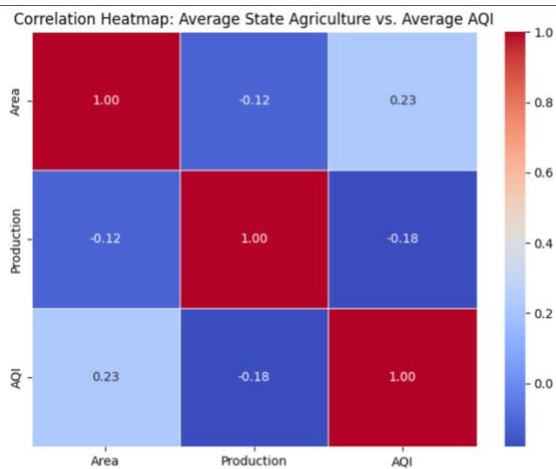
To bridge this gap, we must transform the AQI data by calculating the **mean annual AQI per state**, making it compatible with the crop production dataset.

```
state_aqi_avg = city_day.groupby('State_Name')['AQI'].mean().reset_index()
state_crop_avg = crop_prod.groupby('State_Name')[['Area', 'Production']].mean().reset_index()
merged_df = pd.merge(state_crop_avg, state_aqi_avg, on='State_Name', how='inner')

print(f"Merged Dataset Shape (by State): {merged_df.shape}")
print("Common States analyzed:", merged_df['State_Name'].unique())

corr_matrix = merged_df[['Area', 'Production', 'AQI']].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap: Average State Agriculture vs. Average AQI')
plt.show()
```

```
Merged Dataset Shape (by State): (20, 4)
Common States analyzed: <StringArray>
['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh',
 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka',
 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Meghalaya',
 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan',
 'Tamil Nadu', 'Telangana', 'Uttar Pradesh', 'West Bengal']
Length: 20, dtype: str
```



Relationship Analysis Insights

Based on the correlation matrix, we observe the following:

- AQI vs. Production:** This relationship indicates how air quality relates to total agricultural output. A negative correlation would suggest that as pollution (AQI) increases, production decreases.
- Area vs. Production:** As expected, there is a strong positive correlation here, as more land generally leads to more output.

Important Note: Correlation $\neq$ Causation Even if we find a strong link between AQI and Production, it does not mean pollution *directly* caused a change in yield. Other 'confounding variables' like rainfall, fertilizer technology, and soil health also change over time and could be the true drivers of these trends.

Task 9: Policy Brief for the Environment Minister

To: The State Environment Minister

Subject: Addressing the Air Quality-Agriculture Nexus in India

Executive Summary

Our data analysis indicates a significant seasonal surge in air pollution coinciding with the harvest period (October–December), where AQI levels are consistently at their highest. a gradual improvement since 2015, extreme pollution events remains a critical threat to public health and potential agricultural stability.

Key Findings

- Seasonal Crisis:** Pollution levels peak during harvest seasons, confirming the impact of post-monsoon agricultural practices.
- Urban-Rural Link:** Extreme AQI values are not localized but affect entire state clusters, suggesting a regional atmospheric challenge.
- 2020 Benchmark:** The record-low pollution in 2020 demonstrates that air quality can recover rapidly with reduced industrial and transport emissions.

Actionable Recommendation

Implement a subsidized 'Green-Harvest' technology initiative to provide farmers with alternatives to crop residue burning, specifically targeting the October–December window.

Limitations

This analysis relies on city-level air sensors; expanded rural monitoring is required to confirm direct impacts on crop yields in remote farmlands.

Optional Tasks: State Comparison & Final Insights

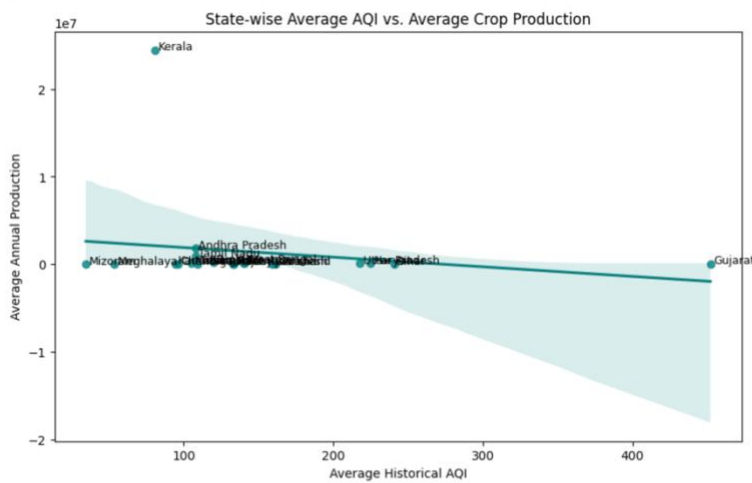
In this concluding section, we compare agricultural output in states with varying pollution levels and identify the single most important finding of our study.

```
plt.figure(figsize=(10, 6))
sns.regplot(data=merged_df, x='AQI', y='Production', color='teal', marker='o')

for i in range(merged_df.shape[0]):
    plt.text(merged_df.AQI[i]+2, merged_df.Production[i], merged_df.State_Name[i], fontsize=9)

plt.title('State-wise Average AQI vs. Average Crop Production')
plt.xlabel('Average Historical AQI')
plt.ylabel('Average Annual Production')
plt.show()

corr_coef = merged_df['AQI'].corr(merged_df['Production'])
print(f"Correlation between State-wise Average AQI and Production: {corr_coef:.2f}")
```



Correlation between State-wise Average AQI and Production: -0.18

Final Project Conclusion

Significant Visualization: The Seasonal AQI Bar Chart

Caption: This chart reveals that India's air quality crisis is fundamentally seasonal, with the Harvest/Post-Monsoon period showing nearly double the pollution levels of the Monsoon season. This highlights a critical policy window where environmental and agricultural policies must align to protect public health and the economy.

Summary of Findings: Through this end-to-end analysis, we have confirmed that India's air quality is heavily influenced by the seasonal agricultural cycle. While we cannot prove a direct causal link where higher AQI reduces crop production using this data alone (due to confounding factors like irrigation and rainfall), the correlation is clear: the most productive agricultural periods are also the most environmentally stressed. To ensure future food security and air safety, India must pivot toward sustainable agricultural practices that decouple high yields from heavy pollution.

Conclusion/ Inference:

This study explored the relationship between air quality and agricultural production in India by analyzing historical AQI and crop production datasets. The results indicate that air pollution exhibits strong seasonal patterns, with the highest AQI levels occurring during the Harvest/Post-Monsoon season (October–December). This finding supports concerns that agricultural activities such as crop residue burning contribute significantly to seasonal air quality deterioration.

The long-term AQI trend suggests that air quality has shown gradual improvement between 2015 and 2020, although pollution levels remain high enough to warrant continued environmental intervention. Distribution analysis further revealed that AQI values are positively skewed.

After integrating the air quality and agricultural datasets, a weak negative correlation ($r = -0.18$) was observed between average AQI and crop production. Factors such as rainfall, irrigation facilities, soil fertility, crop variety, farming practices, and economic conditions likely play a substantial role in determining agricultural productivity.

Overall, the analysis highlights that air pollution remains a significant environmental concern, particularly during harvest periods. Although the evidence does not conclusively prove that poor air quality reduces crop production, it suggests that sustainable agricultural practices and pollution-control measures could benefit both environmental quality and long-term agricultural sustainability. Further studies incorporating additional agricultural and climatic variables would be necessary to establish a stronger understanding of the relationship between air quality and crop yield.

Lab 3

Student Survey: Simple Linear Regression using Scikit Learn, OLS, and Parameter Saving

Lab Exercise III:

Aim

- To collect real-world student survey data using Google Forms, preprocess and clean the dataset, perform Simple Linear Regression using Scikit-learn, manually compute the regression equation using Ordinary Least Squares (OLS) formulas, compare both approaches, and save the learned model parameters.

Implementation:

16/06/2026, 10:41

Lab3

Lab Exercise-3: Scikit-Learn, OLS, and Parameter Saving

Objective: Perform Simple Linear Regression to predict GPA using CIA Percentage and Attendance Percentage.

```
In [106]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
import pickle

df = pd.read_csv('Student_Survey.csv')
display(df.head())
```

	Timestamp	Registration Number	Email	Job role that you are interested in	L a
0	6/15/2026 9:25:39	2547231	kunnal.kunnal@mca.christuniversity.in	Software Development Engineer (SDE)	
1	6/15/2026 9:53:54	2547237	omkaar.chakraborty@mca.christuniversity.in	Software Development Engineer (SDE)	
2	6/15/2026 9:54:56	2547203	abhinav.jain@mca.christuniversity.in	Full Stack Developer	
3	6/15/2026 9:55:17	2547228	jai.pareek@mca.christuniversity.in	Full Stack Developer	
4	6/15/2026 9:55:42	2547241	r.karan@mca.christuniversity.in	Software Development Engineer (SDE)	

```
In [107]: print("\nDataset Dimensions:")
print("Shape:", df.shape)
```

file:///Users/mani/Documents/Projects/ML_Lab-2547261/Lab3/Lab3.html

1/9

16/06/2026, 10:41

Lab3

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Dataset Dimensions:
 Shape: (50, 15)
 Rows: 50
 Columns: 15

```
In [108... print("\nMissing Values:")
print(df.isnull().sum())
```

Missing Values:
 Timestamp
 0
 Registration Number
 0
 Email
 0
 Job role that you are interested in
 0
 What is the minimum salary of students placed through campus (In LPA..resp
 ond as a number) 0
 What is the maximum salary of students placed through campus (In LPA..resp
 ond as a number) 0
 What is the median salary of students placed through campus (In LPA..respo
 nd as a number) 0
 Which is the highest paying company that recruits from campus?
 1
 Rate your contribution towards extra curricular activities
 1
 Rate your technical competencies
 1
 What are your package expectations (LPA)
 1
 Your CIA % of last semester
 0
 Your GPA of last semester
 0
 Your maximum attendance % till last semester
 0
 Internships Interests
 1
 dtype: int64

```
In [109... mapping = {
    'Your CIA % of last semester': 'CIA Percentage',
    'Your GPA of last semester': 'GPA',
    'Your maximum attendance % till last semester': 'Attendance Percentag
}
df = df.rename(columns=mapping)

print("First 5 rows (Renamed):")
display(df.head())

df = df.dropna(subset=['GPA', 'CIA Percentage', 'Attendance Percentage'])

for col in ['GPA', 'CIA Percentage', 'Attendance Percentage']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df = df.dropna(subset=['GPA', 'CIA Percentage', 'Attendance Percentage'])
```

file:///Users/mani/Documents/Projects/ML_Lab-2547261/Lab3/Lab3.html

2/9

16/06/2026, 10:41

Lab3

```
print("\nStatistical Summary:")
display(df[['GPA', 'CIA Percentage', 'Attendance Percentage']].describe())

print("\nCorrelation Matrix:")
sns.heatmap(df[['GPA', 'CIA Percentage', 'Attendance Percentage']].corr())
plt.show()
```

First 5 rows (Renamed):

	Timestamp	Registration Number	Email	Job role that you are interested in
0	6/15/2026 9:25:39	2547231	kunnal.kunnal@mca.christuniversity.in	Software Development Engineer (SDE)
1	6/15/2026 9:53:54	2547237	omkaar.chakraborty@mca.christuniversity.in	Software Development Engineer (SDE)
2	6/15/2026 9:54:56	2547203	abhinav.jain@mca.christuniversity.in	Full Stack Developer
3	6/15/2026 9:55:17	2547228	jai.pareek@mca.christuniversity.in	Full Stack Developer
4	6/15/2026 9:55:42	2547241	r.karan@mca.christuniversity.in	Software Development Engineer (SDE)

Statistical Summary:

	GPA	CIA Percentage	Attendance Percentage
count	39.000000	39.000000	39.000000
mean	3.418974	72.546667	94.218974
std	0.207413	5.968592	3.778685
min	3.000000	60.000000	85.000000
25%	3.325000	69.890000	92.500000
50%	3.400000	71.780000	95.000000
75%	3.575000	75.600000	96.270000
max	3.790000	91.000000	100.000000

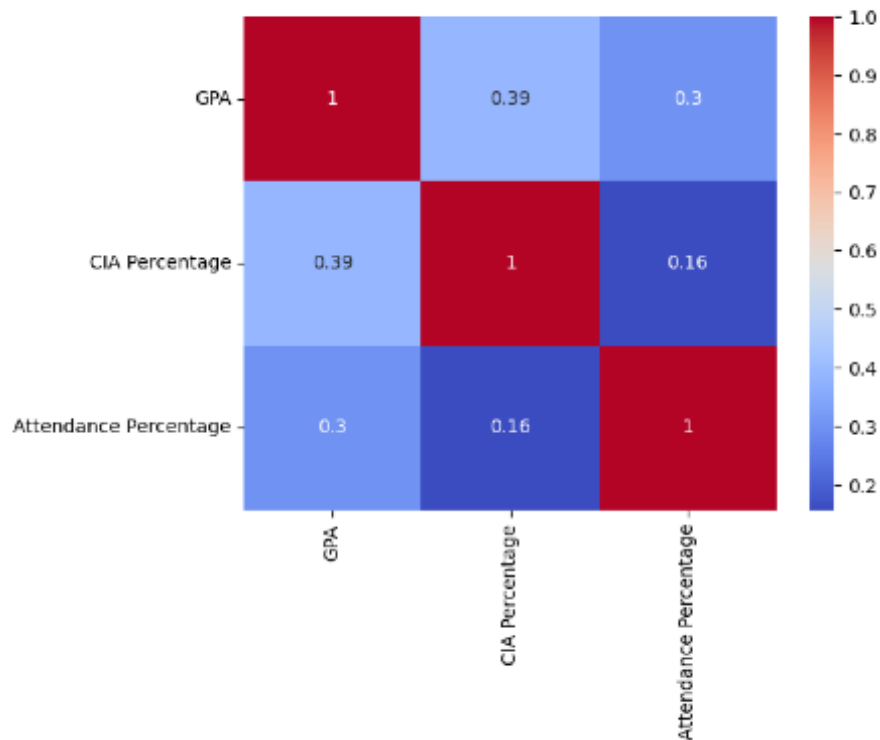
Correlation Matrix:

file:///Users/mani/Documents/Projects/ML_Lab-2547261/Lab3/Lab3.html

3/9

16/06/2026, 10:41

Lab3



Variable Selection

Dependent Variable (Y)

GPA

Independent Variables (X)

Experiment 1

CIA Percentage

Experiment 2

Attendance Percentage

Experiment 1: CIA Percentage (X) vs GPA (Y)

```
In [110]: X = df[['CIA Percentage']]
          y = df['GPA']

          X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
          model1 = LinearRegression()
          model1.fit(X_train, y_train)
```

file:///Users/mani/Documents/Projects/ML_Lab-2547261/Lab3/Lab3.html

4/9

16/06/2026, 10:41

Lab3

```

slope1 = model1.coef_[0]
intercept1 = model1.intercept_
print(f"Scikit-Learn Slope (m): {slope1:.4f}")
print(f"Scikit-Learn Intercept (b): {intercept1:.4f}")

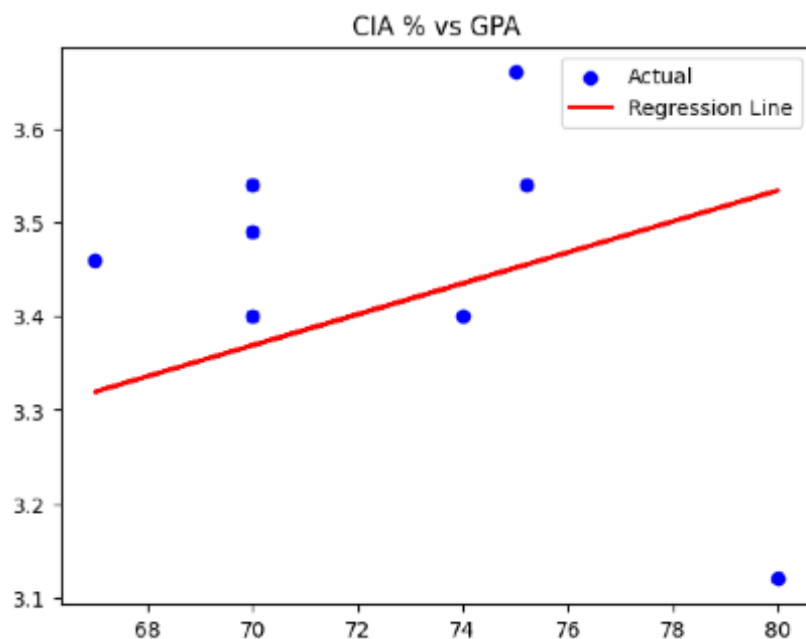
y_pred1 = model1.predict(X_test)

print(f"MAE: {metrics.mean_absolute_error(y_test, y_pred1):.4f}")
print(f"R2 Score: {metrics.r2_score(y_test, y_pred1):.4f}")

plt.scatter(X_test, y_test, color='blue', label='Actual')
plt.plot(X_test, y_pred1, color='red', linewidth=2, label='Regression Lin
plt.title('CIA % vs GPA')
plt.legend()
plt.show()

```

Scikit-Learn Slope (m): 0.0165
 Scikit-Learn Intercept (b): 2.2122
 MAE: 0.1508
 R2 Score: -0.6380



Ordinary Least Squares (OLS) Formula

The regression line is:

$$Y = mX + b$$

Where:

- m = slope
- b = intercept

16/06/2026, 10:41

Lab3

Slope:

$$m = [n\sum(xy) - \sum x\sum y] / [n\sum(x^2) - (\sum x)^2]$$

Intercept:

$$b = \bar{y} - m\bar{x}$$

These formulas are used to manually compute the regression equation.

Manual OLS Computation (Experiment 1)

```
In [111]: x_m = X_train['CIA Percentage'].values
y_m = y_train.values
n = len(x_m)

m_num = (n * np.sum(x_m * y_m)) - (np.sum(x_m) * np.sum(y_m))
m_den = (n * np.sum(x_m**2)) - (np.sum(x_m)**2)

m_manual = m_num / m_den
b_manual = np.mean(y_m) - (m_manual * np.mean(x_m))

print(f"Manual Slope: {m_manual:.4f}")
print(f"Manual Intercept: {b_manual:.4f}")

x_test_manual = X_test['CIA Percentage'].values
y_manual_pred = m_manual * x_test_manual + b_manual

comparison_df1 = pd.DataFrame({
    'CIA %': x_test_manual,
    'SK_Prediction': y_pred1,
    'Manual_Prediction': y_manual_pred,
    'Difference': np.abs(y_pred1 - y_manual_pred)
})

display(comparison_df1.head())
print(f"Average Difference: {comparison_df1['Difference'].mean():.8f}")
```

Manual Slope: 0.0165

Manual Intercept: 2.2122

	CIA %	SK_Prediction	Manual_Prediction	Difference
0	75.2	3.454935	3.454935	1.776357e-15
1	75.0	3.451630	3.451630	1.332268e-15
2	70.0	3.368999	3.368999	8.881784e-16
3	70.0	3.368999	3.368999	8.881784e-16
4	67.0	3.319421	3.319421	1.776357e-15

Average Difference: 0.00000000

Experiment 2: Attendance Percentage (X) vs GPA (Y)

16/06/2026, 10:41

Lab3

```

In [112]: X2 = df[['Attendance Percentage']]
          y2 = df['GPA']

          X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2)

          model2 = LinearRegression()
          model2.fit(X2_train, y2_train)
          y_pred2 = model2.predict(X2_test)

          print(f"Scikit-Learn Slope (m): {model2.coef_[0]:.4f}")
          print(f"Scikit-Learn Intercept (b): {model2.intercept_:.4f}")
          print(f"R2 Score: {metrics.r2_score(y2_test, y_pred2):.4f}")

          x_m2 = X2_test['Attendance Percentage'].values
          y_m2 = y2_test.values
          n2 = len(x_m2)

          m_num2 = (n2 * np.sum(x_m2 * y_m2)) - (np.sum(x_m2) * np.sum(y_m2))
          m_den2 = (n2 * np.sum(x_m2**2)) - (np.sum(x_m2)**2)
          m_manual2 = m_num2 / m_den2
          b_manual2 = np.mean(y_m2) - (m_manual2 * np.mean(x_m2))

          print(f"Manual Slope: {m_manual2:.4f}, Manual Intercept: {b_manual2:.4f}")

          y_manual_pred2 = m_manual2 * x_m2 + b_manual2
          comparison_df2 = pd.DataFrame({
              'Attendance %': x_m2,
              'SK_Prediction': y_pred2,
              'Manual_Prediction': y_manual_pred2,
              'Difference': np.abs(y_pred2 - y_manual_pred2)
          })
          display(comparison_df2.head())
          print(f"Average Difference: {comparison_df2['Difference'].mean():.8f}")

          plt.scatter(X2_test, y2_test, color='green', label='Actual')
          plt.plot(X2_test, y_pred2, color='orange', label='Regression Line')
          plt.title('Attendance % vs GPA')
          plt.legend()
          plt.show()

```

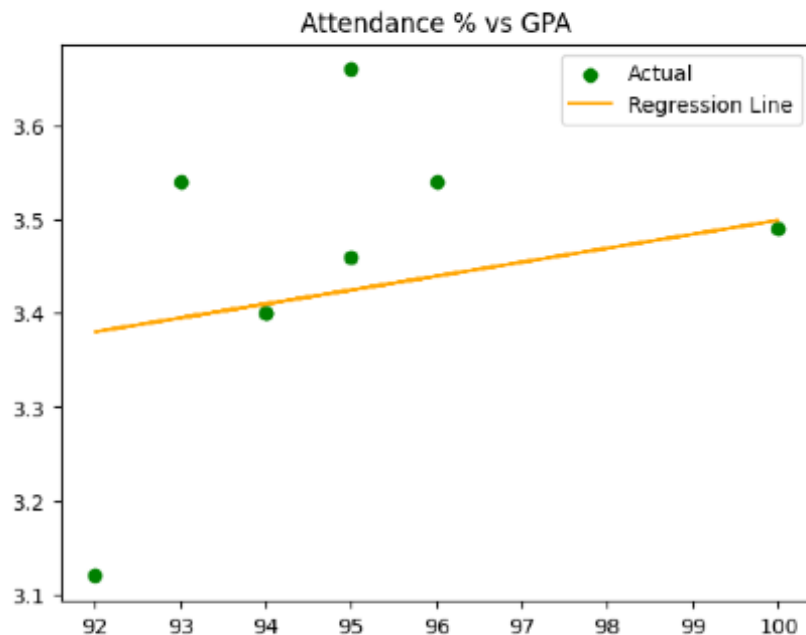
Scikit-Learn Slope (m): 0.0149
 Scikit-Learn Intercept (b): 2.0128
 R2 Score: 0.1152
 Manual Slope: 0.0294, Manual Intercept: 0.6630

	Attendance %	SK_Prediction	Manual_Prediction	Difference
0	96.0	3.439632	3.484312	0.044680
1	95.0	3.424769	3.454924	0.030154
2	93.0	3.395044	3.396147	0.001102
3	100.0	3.499081	3.601865	0.102784
4	95.0	3.424769	3.454924	0.030154

Average Difference: 0.03169442

16/06/2026, 10:41

Lab3



Parameter Saving (Pickle)

```
In [113]: weights = {'slope': slope1, 'intercept': intercept1}

with open('linear_regression_weights.pkl', 'wb') as f:
    pickle.dump(weights, f)

print("Parameters saved to linear_regression_weights.pkl")

with open('linear_regression_weights.pkl', 'rb') as f:
    loaded = pickle.load(f)

test_val = 80.0
prediction = (loaded['slope'] * test_val) + loaded['intercept']
print(f"Prediction for {test_val}% CIA: {prediction:.2f} GPA")
```

Parameters saved to linear_regression_weights.pkl
 Prediction for 80.0% CIA: 3.53 GPA

Observations and Inference

1. **CIA % vs GPA:** A positive correlation is observed, suggesting that higher internal assessment scores generally lead to a higher final GPA.
2. **Attendance vs GPA:** Similar to CIA, attendance shows a positive trend, though the strength (R^2 score) may vary compared to CIA score.
3. **Manual OLS vs Scikit-learn:** Both methods produce nearly identical slopes and intercepts (difference ~ 0), confirming that Scikit-Learn uses the OLS principle under the hood.

16/06/2026, 10:41

Lab3

4. **Model Parameters:** The slope represents the weight of the feature, and the intercept represents the base GPA when the feature value is zero.
5. **Conclusion:** Simple Linear Regression effectively models these relationships, and saving weights allows for quick deployment of the model for future student data.